

User guide for pseudo-dynamic STATE ESTIMATION (SENSEI)

5 de diciembre de 2025

Índice

1. Operating scheme	2
1.1. Differences with calibration	3
2. .inp file info_File containing the initial information	3
2.1. Intermediate .inp file intermediate_File	3
2.2. Time parameters	3
2.3. Demand patterns	4
2.4. Piezometric head (reservoirs)	4
3. .csv files for reading sensor measurements and/or pseudomeasurements	4
4. Aspects affecting the state estimation	6
4.1. Measurements / pseudomeasurements and weights	6
4.2. Jacobian	7
4.3. Sequential algorithm for pseudo-dynamic state estimation	7
4.4. Default values for the algorithm parameters	9
4.5. Tracking the evolution of control-element status	9
4.6. Control of parameter increments	10
5. .txt configuration file (config_multiperiod_SE.txt)	10
6. .csv files containing the SE results	11

1. Operating scheme

This document explains how to configure the **pseudo-dynamic State Estimation (SE)** tool of *SENSEI*, which is applied to a water distribution network given a time series of measurements and/or pseudomeasurements over a certain period of time. Some details of the explanation correspond to examples performed on the `.inp` consisting of the first 12 hours of the calibrated C-town model, which we call *our_C-town*, to highlight that it is a specific reduced-duration network obtained through a calibration process.

It describes how to include measurement/pseudomeasurement `.csv` files and other aspects required for the correct operation of the tool. All necessary files are located in the ***Multiperiod_SE*** folder. The user is free to use the provided files and even modify the values in the `.csv` files, add or remove measurements, etc., to observe how such changes affect the estimation process.

The ***Multiperiod_SE*** folder is structured as follows:

- **Executable** file `Multiperiod_SE_SENSEI.exe`
- **Configuration** file `config_multiperiod_SE.txt`
- Subfolder ***SE_Examples_our_C-town***, which includes:
 - **info_File**, named `our_C-town0.inp` (**network model**)
 - Subfolder ***NORMAL*** (normal operating situation, only with measurement noise)
 - Subfolder ***BIAS_FM_T3*** (biased water-level sensor at T3, with measurement noise)
 - Subfolder ***LEAK_J59_emitter1*** (leak at node J59, via an emitter coefficient equal to 1.0 added at the eighth hour)
 - Each of the three subfolders contains the measurement and/or pseudomeasurement files **dm_File**, **fm_File**, **pm_File**, and **wlm_File**, with their respective coefficients of variation. The filenames vary slightly from one subfolder to another.
- 13 **dynamic libraries**: `epanet2.dll`, `libamd.dll`, `libcamd.dll`, `libccolamd.dll`, `libcholmod.dll`, `libcolamd.dll`, `libgcc_s_seh-1.dll`, `libgfortran-5.dll`, `libgomp-1.dll`, `libopenblas.dll`, `libquadmath-0.dll`, `libsuitesparseconfig.dll`, and `libwinpthread-1.dll`

The executable `Multiperiod_SE_SENSEI.exe` opens a terminal through which the user can observe the progress of the pseudo-dynamic SE, which runs automatically based on the predefined configuration. The user simply needs to proceed through the pauses between steps by pressing any key when indicated.

Below we detail the modifications that the user is allowed or not allowed to make in order for SE to run properly.

What should the user NOT do?

- Delete any of the dynamic libraries (`.dll` files)
- Change the name of the configuration file `config_multiperiod_SE.txt`, since the executable has been built to read a file with exactly that name.
- Rearrange or, in general, modify the internal structure of the ***Multiperiod_SE*** folder, as this may cause issues related to directories and file locations.

What can the user modify?

- Modify the content of the measurement or pseudomeasurement files, changing existing values, adding or removing rows, etc.

- Change the name of any file within each subfolder (including the subfolder itself), as long as the new name is updated in the configuration file `config_multiperiod_SE.txt`.
- Modify parameters related to the iterative algorithm inside `config_multiperiod_SE.txt`, although it is recommended to keep the default values provided in the file.
- Select English or Spanish as the language for displaying information and messages during the process.

Nevertheless, **any modification must always be carried out following the guidelines indicated later on**, in each specific section.

1.1. Differences with calibration

Performing SE requires a less complex preliminary configuration than calibration, since, *broadly speaking*, the problem only seeks the **most probable value of the state variables (average demands at demand junctions throughout each interval and tank levels at the start of each interval)**, assuming that the network has already been calibrated. In contrast, calibration primarily seeks to find the most probable internal configuration in terms of global parameters (roughness coefficients, pump curves, valve settings, etc.), which change very slowly and can thus be considered constant over short time windows (the ones used for SE). For this reason, SE does not require activating options or adding extra files.

Another major difference between calibration and pseudo-dynamic SE is the simulation duration. In calibration, the system is always simulated over the entire set of T periods; whereas in multiperiod SE, as a sequential process, all simulations last only one period. When the adjustment for one period is completed, the algorithm moves to the next, and so on. Therefore, **SE is a considerably more manageable problem**, given its much lower computational burden.

2. .inp file info_File containing the initial information

The .inp file referred to as `info_File` will serve as the **reference model for the state estimation**, since it is used to store data related to the **network topology**, **count the number of elements** of each type, and **store key positions** of certain elements. In addition, it defines key parameters that configure the SE process.

→ For more information on how to modify the .inp file in a way that is compatible with EPANET, **you can consult this [website](#)**.

2.1. Intermediate .inp file intermediate_File

The reference .inp file `info_File` **will never be modified**. All changes applied to the network during SE will be made on an intermediate file, `intermediate_File`, which will be deleted once the state estimation process concludes (but not if it is aborted midway).

→ If the file is `info_File = our_C-town.inp`, the intermediate file will be named `intermediate_File = our_C-town_intermediate.inp`, and it will be located in the ***SE_Examples_our_C-town*** subfolder.

2.2. Time parameters

In the [TIMES] section of `info_File`, the user must specify the time-related parameters `DURATION`, `HYDRAULIC TIMESTEP`, and `PATTERN TIMESTEP`, which determine aspects of the simulation related to simulation time and the dimensions of matrices, vectors, etc.

- **DURATION**: duration of the time series of measurements/pseudomeasurements, which will be the total (accumulated) duration of the process.

- **HYDRAULIC TIMESTEP**: frequency at which equations are recalculated (provided no event occurs earlier).
- **PATTERN TIMESTEP**: frequency of the collected data; equivalently, the time interval between one data point and the next. Ultimately, it determines the **duration of each interval or period**, and therefore, the duration of each simulation.

Important considerations for time parameters

- Once **DURATION** and **PATTERN TIMESTEP** are specified, the number of periods T for calibration is determined as their quotient; therefore, **DURATION** must be a multiple of **PATTERN TIMESTEP**.

$$T = \frac{\text{DURATION}}{\text{PATTERN TIMESTEP}} = \text{number of periods for which data are available}$$

- Furthermore, it is advisable that **HYDRAULIC TIMESTEP** be a divisor of **PATTERN TIMESTEP**, although both parameters may also be equal.

For the *our_C-town* case, we defined **DURATION** = 12:00 (12 hours) and **PATTERN TIMESTEP** = 1:00 (i.e., hourly records), which produces a sequential estimation with $T = 12$ one-hour periods. In addition, **HYDRAULIC TIMESTEP** was set to 0:10, meaning that equations are recalculated every 10 minutes.

→ Once work begins on the **intermediate_File**, **DURATION** = **PATTERN TIMESTEP** will automatically be set, reducing the simulation time to the duration of a single period.

2.3. Demand patterns

Unlike the configuration required for calibration, SE imposes no restriction on the length of demand patterns. For any demand pattern in the **info_File** model, its length will be reduced to 1 and its (only) component will be set to 1. In this way, since we work with independent periods, the net demand will be modeled exclusively through the base demand, ignoring the multiplier factor (which becomes irrelevant when equal to 1).

2.4. Piezometric head (reservoirs)

As in calibration, SENSEI's state estimation does not treat the piezometric head of a reservoir as a variable to optimize. It is considered a known value, specified by the user in **info_File**. A brief reminder of available options:

- If a constant piezometric head is assumed for a reservoir, it is sufficient to specify the value in the **.inp** (this is the usual approach).
- If, on the contrary, one wishes to prescribe a different head for each interval, a piezometric head pattern may be assigned to the reservoir. When reading the **.inp**, its values will be stored in a structure to *impose* the corresponding value in each interval.

3. .csv files for reading sensor measurements and/or pseudomeasurements

These files contain the information collected by sensors measuring:

- Demand at **all** demand nodes, in **dm_File**. The average demand consumed over the interval must be provided.
- Water level of **all** tanks (NOT reservoirs), in **wlm_File**. The tank level at the beginning of each interval must be provided, and additionally at the end of the final interval.
- Pressure at junctions, in **pm_File**. The pressure at the beginning of each interval must be provided.

- Instantaneous flow, for links equipped with sensors that record pointwise flow measurements, in `fm_File`. The instantaneous flow at the beginning of each interval must be provided.
- Averaged flow, for links with sensors that record flow based on accumulated volume over a time period, in `fm_avg_File`. The average flow during the interval must be provided.

Each file must be structured as follows:

Structure of the sensor measurement `.csv` files

- **Each row** corresponds to **one sensor**.
- Regarding the **columns**:
 - 1st column: **ID of the element on which the sensor is located**, which must match an element in `info_File`.
 - For `dm_File`, the ID must match that of a demand junction (those with nonzero base demand).
 - For `wlm_File`, the ID must match that of a tank.
 - For `pm_File`, the ID must match that of a junction.
 - For `fm_File` and `fm_avg_File`, the ID must match that of a link (including pipes, pumps, and valves).
 - 2nd column: **coefficient of variation (CV) or measurement error rate**. This is the percentage deviation allowed from the true value, expressed as a number between 0 and 1.
 - In the remaining columns, the **measured values** must be provided, ordered by instant.

→ **EXTRA**: If some measurements are suspected to be less accurate than others at certain instants, a **CV different from the general one may be specified in an extra row**, writing the distinct CV directly below the measurement it corresponds to. For example:

	A	B	C	D	E	F	G	H
1	T1	0.01	3	2.82	2.7	2.62	2.7	2.89
2	T2	0.02	0.5	0.72	0.96	1.19	1.57	2.37
3					0.1	0.2	0.2	
4	T3	0.015	3	3.51	4.05	4.6	5.19	5.15

In the figure above, measurements from 3 tank-level sensors are shown at 6 time instants (for example, corresponding to an SE process with 5 periods, where each value corresponds to the start of each interval and the last value corresponds to the level at the end of the final period). Each sensor has a different CV. For tank T2, a CV different from the general one is specified at the third, fourth, and fifth instants.

Optional files and mandatory files

- The files `pm_File`, `fm_File`, and `fm_avg_File` are **optional**. If no sensors of a given type are available, there is no need to prepare an empty `.csv` file.
 - If a cell is left blank, it is assumed that the measurement at that instant is missing, and a zero weight will be assigned.
- The files `dm_File` and `wlm_File` are **mandatory**. The reason is that, being **state variables** in each period's adjustment, EPANET requires their values to perform a simulation.

- If no sensor values are available for some demand or some tank level, a **pseudomeasurement** must be specified—a rudimentary estimate or a historical/expected value, with a high CV (e.g., between 0.1 and 0.3).
- Unlike the pressure and flow measurement files (instantaneous and averaged), these files **must have all entries filled**. If a sensor fails at any instant, the corresponding cell cannot be left blank and must be filled with a pseudomeasurement.
- When reading `dm_File` and `wlm_File`, the program checks that there are as many rows as demand nodes and tanks, respectively, and also that no cell is empty. Otherwise, the user is required to fix the `.csv` file.

As a summary and to clearly state which measurements are averaged and which are not, and the instant to which each corresponds, **it is recommended to keep the following table as a reference:**

	3rd col. Period [0h, 1h)	4th col. Period [1h, 2h)	...	14th col. Period [11h, 12h)	15th col. Instant 12h
<code>dm_File</code>	Avg. dem. [0h, 1h)	Avg. dem. [1h, 2h)	...	Avg. dem. [11h, 12h)	-
<code>wlm_File</code>	Level at 0h	Level at 1h	...	Level at 11h	Level at 12h
<code>pm_File</code>	Pressure at 0h	Pressure at 1h	...	Pressure at 11h	-
<code>fm_File</code>	Flow at 0h	Flow at 1h	...	Flow at 11h	-
<code>fm_avg_File</code>	Avg. flow [0h, 1h)	Avg. flow [1h, 2h)	...	Avg. flow [11h, 12h)	-

Cuadro 1: Schema for entering information in measurement/pseudomeasurement `.csv` files for a 12-hour SE process with hourly data. Note that the `.csv` must be filled starting from the 3rd column, since the 1st one corresponds to the ID and the 2nd one to the CV, as explained above.

4. Aspects affecting the state estimation

This section provides a summarized description of how certain components of SENSEI's pseudo-dynamic state estimation process work, in order to give the user a general idea of how the choice of certain parameters or *inputs* may influence the final estimation result. In addition, it explains how some of these values can be modified through the configuration file.

4.1. Measurements / pseudomeasurements and weights

Reading the measurement and pseudomeasurement `.csv` files provides, for each defined element and each instant (or interval), a measurement z_i with an associated coefficient of variation CV_i . This allows us to define a standard deviation for the measurement/pseudomeasurement as $CV_i \cdot z_i$. The weight corresponding to z_i , denoted w_i , is defined as the inverse of the measurement variance:

$$w_i = \frac{1}{(CV_i \cdot z_i)^2} \quad (1)$$

Notes on the weights

- The lower the variability of a measurement, the higher our confidence in it and the greater its weight; and vice versa. The weight (relative to the measured value) indicates the *importance* that the variable has in the objective function.

- If $z_i = 0$ (the measured value is zero; for example, instantaneous flow that momentarily reads 0 because no water is flowing through the pipe), then SENSEI automatically sets $w_i = 0.05$ to avoid division by zero in (1).
- If a measurement is missing at a given instant (a blank cell in the `.csv` file), the weight is set to zero, $w_i = 0$, so that the variable does not affect the objective function.

4.2. Jacobian

The Jacobian matrix is the key component of the optimization algorithm. It contains the derivatives of the measurements (*rows*) with respect to the state variables (*columns*) of each period.

Numerical computation of the Jacobian

SENSEI computes the derivatives numerically:

- In each iteration, we start with the vector $g(\mathbf{x})$, containing the measurement/pseudomeasurement values obtained through simulation for a set of state variables $\mathbf{x} = (x_1, \dots, x_n)$.
- The variable with respect to which the derivative is to be computed (say, the j -th variable) is perturbed by $\varepsilon\%$, producing the perturbed vector $\mathbf{x}_\varepsilon = (x_1, \dots, x_j(1 + \varepsilon/100), \dots, x_n)$, and the simulation is run again, obtaining $g(\mathbf{x}_\varepsilon)$.
- The derivative vector (the entire j -th column) is then computed as

$$\frac{g(\mathbf{x}_\varepsilon) - g(\mathbf{x})}{x_j \cdot \varepsilon/100} \quad (2)$$

→ For the derivative computation in (2) to be faithful to the model, the perturbation must be *small*. Therefore, the default perturbation rate is $\varepsilon = 0.2$, although this can be modified later.

4.3. Sequential algorithm for pseudo-dynamic state estimation

Within each period's adjustment, we seek the most probable values of the state variables (average demands during the interval and tank levels at its beginning) given the available information, which is divided as follows:

- At the beginning: initial tank levels (measurements/pseudomeasurements in all tanks), pressure (measurements at junctions equipped with pressure sensors), and instantaneous flow (measurements at links equipped with sensors).
- During the interval: average demands (measurements/pseudomeasurements at all demand junctions) and averaged flows (measurements in links with corresponding sensors).
- At the end: tank levels at the end of the interval (measurements/pseudomeasurements in all tanks, coinciding with the initial levels of the next interval).

For each period, the search for the optimal state variables will iterate at most `max_it` times before moving on to the next period.

In each iteration, we solve the **Levenberg–Marquardt** equation:

$$(J_k^t \cdot W \cdot J_k + \lambda \cdot D_k^t \cdot D_k) \cdot \Delta \mathbf{x}_{k+1} = J_k^t \cdot W \cdot (z - g(\mathbf{x}_k)), \quad (3)$$

where $D_k^t \cdot D_k$ is a diagonal matrix containing the diagonal elements of $J_k^t \cdot W \cdot J_k$.

→ Note that a **regularization term**, λ , is included. For each period, SENSEI starts with an initial value λ_0 .

- If, on the first attempt (the first solution of (3)), the objective function does not improve, λ is increased (i.e., more regularization), and (3) is solved again. This is repeated up to `max_attempts` times. If all attempts are exhausted, SENSEI terminates the adjustment of the current period and proceeds to the next one.
- Otherwise, if the objective function improves, the variables are updated, λ is reduced (i.e., less regularization), and the adjustment continues until convergence or until `max_it` iterations have been reached.

Algorithm pseudocode

```

1 for t in 1:T: #for each period
2     x0 ← initial state of the network #initial values of the state
        variables
3     if t > 0:
4         * Enforce final control-element status from previous period
5         * Update measurements and weights for initial tank levels
6     Compute residuals r = z - g(x0)
7     F(x0) = r^t*W*r #initial objective function value
8     SWSE ← F(x0)
9     lambda ← lambda_init #initialize lambda for each period
10
11     for k in 1:max_it:
12         attempts ← 0
13         break_while ← 0
14         x_k ← state at iteration k
15
16         Compute J(x_k)
17
18         while break_while == 0:
19             Solve: (J^t*W*J + λ*D^t*D) Δx = J^t*W*r
20             x_prov ← x_k + Δx
21             Compute residuals r = z - g(x_prov)
22             F(x_prov) = r^t*W*r
23             SWSE_prov ← F(x_prov)
24
25             if SWSE_prov < SWSE:
26                 x_{k+1} ← x_prov #update variables if improvement
27                 SWSE ← SWSE_prov
28                 lambda ← lambda / 10
29
30                 if convergence:
31                     break_while ← 1
32             else:
33                 lambda ← 10 * lambda // Increase lambda and retry
34                 attempts += 1
35                 if attempts > max_attempts:
36                     break_while ← 1
37

```

```

38      * Propagate uncertainty and compute normalized residuals
39      * Store control-element status

```

Note: $SWSE = \text{Sum of Weighted Squared Errors}$

Let $\mathbf{x}_{\text{new}}$ and $\mathbf{x}_{\text{old}}$ denote the newly updated variables and their previous values, respectively. Convergence is evaluated through the rate:

$$\frac{\|g(\mathbf{x}_{\text{new}}) - g(\mathbf{x}_{\text{old}})\|_1}{\|g(\mathbf{x}_{\text{new}})\|_1}$$

If this rate is smaller than the tolerance `tol` defined at the beginning, the algorithm is considered to have converged and moves to the adjustment of the next period.

4.4. Default values for the algorithm parameters

Default values for the algorithm parameters

- Perturbation rate for the Jacobian computation: $\varepsilon = 0.2$
- Initial regularization parameter at each period: $\lambda_0 = 0.01$
- Maximum number of iterations at each period: `max_it` = 5
- Maximum number of attempts increasing λ for solving (3): `max_attempts` = 5
- Tolerance: `tol` = 1e-5

→ These algorithm parameters can be modified in the configuration file (explained in a later section), but **it is recommended to retain the default values**.

4.5. Tracking the evolution of control-element status

The **status of control elements** (pipes than can be closed, pumps and/or valves whose status may change based on a control rule defined in `info_File`) **is not a variable to be optimized** in the proposed adjustment.

The pseudo-dynamic SE approach provides, for each period, an estimation of values and statuses at its end. At the optimum, **SENSEI stores the final status of the control elements** (simulated based on the most probable state variables) and **forces it as the initial status of the next period** to maintain continuity.

For all these reasons, it is **crucial** that the user:

- **Provides a correct initial status** for each control element in the `info_File`.
- **Introduces control rules correctly**:
 - If they depend on tank levels, it is essential that the model is well calibrated.
 - If they respond to a status change at a specific time, it must be represented in the `.inp` via a time-based control rule, e.g., 'LINK PU2 CLOSED AT 8:15:55' if it is known that the valve was closed at that time.

4.6. Control of parameter increments

Solving (3) provides an increment for the state variables being optimized, $\Delta \mathbf{x}_{k+1}$. The provisional variables are computed by adding this increment to the current variables:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_{k+1} \quad (4)$$

However, (4) may produce undesired values that are physically impossible:

- For initial tank levels, if the newly estimated value exceeds the maximum level or falls below the minimum, it is bounded accordingly.
- For demands, negative demand values are not allowed. In such cases, the value is clipped to 0.

5. .txt configuration file (config_multiperiod_SE.txt)

In this file, named `config_multiperiod_SE.txt` and **whose name cannot be changed**, the user must specify the language in which information and messages will be displayed, the name and location of the files used for SE, and the values of the algorithm's hyperparameters. An example is:

```
1 LANGUAGE (ENG/ESP); ENG
2 -----
3 PATH; SE_Examples_our_C-town
4 -----
5 info_File; our_C-town.inp
6 dm_File; NORMAL/dm_our_C-town.csv
7 wlm_File; NORMAL/wlm_our_C-town.csv
8 pm_File; NORMAL/pm_our_C-town.csv
9 fm_File; NORMAL/fm_our_C-town.csv
10 fm_avg_File; -
11 -----
12 epsilon; -
13 max_it; -
14 tol; -
15 lambda_init; -
16 max_attempts; -
17 emitter_ID; -
18 emitter_coef; -
19 period_to_add_emitter; -
```

Considerations regarding the .txt configuration file

- The **language** may be **English or Spanish**, activated by specifying '**ENG**' or '**ESP**', respectively, in the field LANGUAGE. If the value is left empty or any other word is written, English will be activated by default.
- It is **essential** that the measurement/pseudomeasurement .csv files and the .inp info_File be placed inside a **subfolder**, indicated in PATH.
- It is **mandatory to specify a calibrated reference model info_File** and a **measurement/pseudomeasurement file for the average demands** of each interval at all demand junctions (dm_File), as well as the **tank levels at the beginning of every interval and also at the final instant**, in

wlm_File.

- It is **not necessary to include** pm_File, fm_File, or fm_avg_File if those types of measurements are not available. In such cases, one must write ‘ - ’ (a **dash**), as shown in the example for fm_avg_File.
- To keep the **default values of the algorithm hyperparameters**, simply leave a ‘ - ’ (dash), as in the example above. If any value is to be modified (not recommended initially), the desired value must be specified.
- The fields emitter_ID, period_to_add_emitter, and emitter_coef must remain as dashes if SE is to be performed using the model exactly as given in info_File, without adding leaks beyond those that may already be present in that .inp. **If, however, after performing an adjustment, signs of a leak appear in a specific area (based on normalized residuals), SENSEI allows running a new adjustment**
 - **considering an emitter** with coefficient emitter_coef,
 - at the node suspected of leaking, emitter_ID (e.g., ‘J67’),
 - and from a given moment onward, period_to_add_emitter, starting from 0 (e.g., to introduce the emitter coefficient at the beginning of the fourth period, one must specify 3).
- **IMPORTANT: Do not change** the text on the left-hand side (before the ‘ ; ’), since the executable is built to read it exactly as written in order to store the information correctly.

→ If within the PATH subfolder (in our case, *SE_Examples_our_C-town*) there are additional subfolders that contain the .csv files dm_File, wlm_File, pm_File, etc. (as in the sub-subfolders *NORMAL*, *BIAS_FM_T3*, and *LEAK_J59_emitter1*), then the internal path to those files within PATH must also be specified; for example, *NORMAL/dm_our_C-town.csv*, *NORMAL/wlm_our_C-town.csv*, etc.

In short, one must always provide the correct path to both the .csv files and the .inp file, specifying any subfolders where they may be located inside the main PATH folder.

6. .csv files containing the SE results

As previously mentioned, at the end of the adjustment of each of the T periods, the adjusted values are stored and uncertainty is propagated. Once the adjustment of the final period is completed, all this information is gathered and saved into several .csv files, which constitute the tools provided by SE to detect anomalies and problems in the network.

.csv files containing the SE results

Given the optimal state variables $\mathbf{x}^*$ (average demands and initial tank levels), $g(\mathbf{x}^*)$ can be computed via simulation. This corresponds to the variables that are compared against the measured values $\mathbf{z}$: the state variables themselves, pressures, instantaneous and/or averaged flows (if such measurements exist), and tank levels at the end of the interval.

In each .csv file, **each row corresponds to one variable** and **each column corresponds to one period**.

The files generated at the end of the process will be saved in PATH (in our case, *SE_Examples_our_C-town*). Additionally, to avoid overwriting files from previous adjustments, the date and time at which the files are saved are appended as a suffix, in YYYYMMDD and hhmmss format, respectively. Thus, assuming that info_File = *our_C-town.inp* and that the files were saved on 24/12/2025 at 21:35:42, the output files will be:

1. **Estimated values** file: `our_C-town_estimated_values_20251224_213542.csv`
2. **Normalized residuals** file: `our_C-town_normalized_residuals_20251224_213542.csv`
3. **Posterior CV** file: `our_C-town_posterior_CV_20251224_213542.csv`

Recall that for the adjustment of a new period, the tank-level value at the end of the previous interval replaces (corrects) the initially provided measurement. On the other hand, the initial tank level becomes a state variable in the new adjustment. For this reason, it is common that the estimated level of a tank at the end of an interval (in one column) does not coincide with the estimated level of that same tank at the beginning of the next period (in the following column, since it is now a variable of the next adjustment).